Team Task System

Practical exercise — React Front-End Developer

WEBNS Technology Ltd.

Before you start

Thanks for taking the time to do this.

This brief describes a **problem**, not a specification. We've told you what the product needs to do and left the rest to you: the data model, the screens, the workflow, the technology. Those decisions are the point. Two candidates who both "finish" this will hand in very different things, and the difference is most of what we're assessing.

How long to spend. Around four hours on the frontend. If you also build a backend, don't let the total run past six. We're not measuring stamina, and we'd rather see a narrow thing built well than a wide thing built badly. Deciding what to leave out is part of the exercise — tell us what you cut and why.

AI tooling is allowed. Copilot, Claude, Cursor, whatever you normally use. Tell us in the README what you used it for. The only condition is that you can explain and change every line you submit, because the follow-up call involves doing exactly that in front of us.

Deadline: five days from the day you receive this brief. If you need longer, message us and say so — we'd rather move the date than get a rushed submission.

The problem

A team of eight to fifteen people share a body of work. Right now it lives in a spreadsheet and nobody trusts it.

They need a tool where:

- Anyone can see what the team is working on, and narrow it down to what matters to them right now
- A specific piece of work can be found in seconds, by name or by who owns it
- Work moves through stages as it progresses, and moving it is quick
- New work can be added without ceremony
- What's urgent, overdue, or unassigned is obvious without reading every row
- A filtered view can be sent to a colleague, and they see the same thing
- All of the above works on a phone, because half of it happens between meetings

That's the brief. Build the frontend for it.

What we are deliberately not telling you

We are not going to specify the data model, the fields, the workflow stages, the number of screens, or the layout. We're not giving you sample data or an API either.

You decide:

- What a piece of work actually consists of, and which fields earn their place
- What the workflow stages are and how many there should be
- Whether this is a table, a board, a list, or something else — and why
- How filtering and search present themselves, particularly on a small screen
- What belongs on the first screen and what can wait behind a click
- Where your scope ends

Every one of those is a judgement call with better and worse answers, and we'll ask you about them. A sparse, opinionated product with a clear rationale scores higher here than a feature-complete one that just did everything the brief implied.

Non-negotiables

Everything above is open. These are not.

React with TypeScript. Everything else in the stack is your choice.

The UI is the deliverable. This is a front-end role and the interface is what we're primarily judging. It should look like a product someone designed, not the default output of a framework — a real type scale, a spacing rhythm, a colour system where status and urgency read at a glance, and density suited to a tool people scan rather than a marketing page. Restraint reads as more senior than decoration.

Fully responsive. It must work at 375px, 768px, and 1280px, and everywhere between. We will resize the window, and it's the first thing we do.

- No horizontal scrolling and no clipped content at any width
- If you build a table, it has to become something genuinely usable on a phone — not a table with a scrollbar attached
- Touch targets comfortable for a thumb
- Filter controls need a real mobile answer; a row of dropdowns is not it
- Nothing jumps around as data loads

Interaction states. Hover, focus-visible, active, and disabled on every interactive element. Loading, error, and empty are three different situations and should look and read differently — one spinner for all three is the easy answer and the wrong one. Errors need a way to retry. Empty states should say why it's empty and what to do about it.

Keyboard and contrast. Someone using only a keyboard should be able to search, filter, and open an item. Text should meet contrast requirements against whatever background you chose.

Shareable views. Search, filters, sort, and pagination belong in the URL. The back button should behave the way a user expects.

No component kit that draws the screen for you. MUI, Ant, Chakra, or a pre-built data grid would answer the exact question we're asking. Headless libraries — Radix, React Aria, Headless UI, TanStack Table's headless core — are welcome, and reaching for one to get the accessibility details right is a good sign.

Your data

You need data. Where it comes from is up to you: hardcoded fixtures, a generated JSON file, MSW, json-server, or a real backend.

How you populate it is part of the assessment. An interface looks effortless with eight tidy rows and falls apart with realistic content, so we'd expect a serious submission to be tested against data that resembles a real backlog:

- Enough volume that pagination or virtualization actually matters — a few hundred items, not a dozen
- Titles of wildly different lengths, including some far longer than your layout comfortably allows
- Records with missing values — no owner, no due date, no description
- A spread of dates including overdue, today, and far in the future
- At least one person whose name is inconveniently long

If we open your app and see twelve neat rows that all fit perfectly, that tells us something about how the layout would hold up in production.

Backend work (optional, and it counts for you)

The frontend is what's being scored. A frontend-only submission with mocked data can score full marks on everything above.

That said, we do work across the stack here, and if you have backend skills we'd like to see them. If you build a real backend, use **PostgreSQL**. What we'd look at:

- A schema that makes sense — sensible types, real constraints, the relationships modelled properly rather than everything flattened into one table
- Filtering, sorting, search, and pagination handled by the database, not by fetching everything and filtering in the browser
- Indexes that match the queries your UI actually makes
- Migrations and a seed script, so the thing can be recreated from scratch

It has to run. If it needs PostgreSQL, include a `docker-compose.yml`, or deploy it somewhere and send a link alongside the repo. If we can't get it running in a few minutes, the backend can't be credited — so if you go this route, make sure the app still demonstrates well if the database isn't available.

Backend work adds to your score. It cannot compensate for a weak interface.

What to send us

A **git repository** — link or zip — with real commit history. One commit called "initial commit" tells us nothing about how you work.

A **README** covering:

- How to run it from a clean clone, including any database setup
- **Screenshots at all three widths.** This is the first thing we look at.
- **Your data model and why.** What a work item consists of, what you left out, what your workflow stages are and why those.
- **Your product decisions.** Layout choice, what you put on the first screen, how the interface adapts on mobile.
- What you decided *not* to build, and why
- The two or three decisions you're least confident about, and the alternatives
- What you used AI tooling for

The README is assessed. Candour about trade-offs reads far better than a claim that everything went smoothly.

Send us a link, not a zip. Push it to GitHub or GitLab and make it public, or share it privately with us on request. If you deployed it anywhere, send that link too — it saves us setup time and works in your favour.

Message the link on **WhatsApp: 01988000841** (+880 1988 000841 from outside Bangladesh) within five days.

WhatsApp is the only channel we use for this, so please don't email. Include your name with the link.

How we'll assess it

Nothing here is hidden. This is the list we work from.

AREA	WEIGHT
Visual craft — hierarchy, typography, colour, consistency	25%
Responsive behaviour across the three widths	20%
Product judgement — data model, scope, what’s on screen and what isn’t	20%
CSS architecture and component design	15%
Interaction states, keyboard access, contrast	10%
Communication — README, screenshots, commits	10%

Plus up to 15 bonus points for backend work: schema design, query handling done in the database, and a project that actually runs from a clean clone.

Things that will **not** count against you: plain CSS over a framework; mocked data rather than a backend; a small feature set you can defend; an unusual layout choice with a reason behind it; telling us something is unfinished.

The one thing that will: a submission you can’t explain.

What happens next

If we move forward, we’ll book a **45-minute call**, split roughly in half.

First you walk us through what you built and we ask about the decisions — why the data model looks like that, what happens to the layout between 1280 and 375, what you cut and what it cost.

Then we’ll ask you to make a change to your own code, live, with us. It isn’t timed and it isn’t a trick; we’ll help if you get stuck. Watching someone work in a codebase they know tells us far more than reading a diff does.

Questions

If something here is ambiguous, ask — or make a decision, write down the assumption, and tell us. Both are good answers. Silently guessing is the weak one.

Message us on **WhatsApp: 01988000841**. We usually reply the same day.

Good luck, and thanks again for your time.